

Os conceitos de desenvolvimento seguro de protocolos e aplicativos são pilares essenciais de uma segurança cibernética robusta. Protocolos como HTTPS, SMTPS e SFTP fornecem canais de comunicação criptografados, que garantem a confidencialidade e a integridade dos dados durante a transmissão. Da mesma forma, protocolos de segurança de email como SPF, DKIM e DMARC funcionam para autenticar identidades de remetentes e proteger contra phishing e spam. As práticas de codificação segura abrangem a validação de entradas para impedir ataques como injeção de SQL ou XSS, aplicando o princípio do privilégio mínimo para minimizar a exposição durante uma violação, implementando o gerenciamento seguro de sessões e atualizando e corrigindo uniformemente os componentes de software. Os desenvolvedores também devem projetar software que gere registros estruturados e seguros para dar suporte a recursos eficazes de monitoramento e alerta.

Nesta lição, você vai fazer o seguinte:

- Compreender conceitos de protocolo seguro.
- Explorar os recursos de filtragem de DNS.
- Revisar protocolos de segurança de emails.
- Explorar técnicas para segurança de aplicativos.
- Revisar conceitos de sandbox (área restrita).



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Protocolos seguros são fundamentais para manter a segurança dos dados. Os protocolos seguros incluem HTTPS (Hypertext Transfer Protocol Secure ou protocolo de transferência de hipertexto seguro) para tráfego seguro na Web, SMTPS e IMAPS para proteger protocolos de email tradicionais, SFTP (SSH File Transfer Protocol ou protocolo de transferência de arquivo SSH) ou FTPS (File Transfer Protocol Secure ou protocolo protegido de transferência de arquivo), LDAPS para acesso seguro a diretórios e DNSSEC para consultas DNS seguras. Esses protocolos garantem que dados confidenciais sejam transmitidos com segurança.

Além disso, existem vários protocolos para ajudar a identificar e mitigar emails de phishing e spam que atuam autenticando as identidades do remetente e verificando a integridade da mensagem, incluindo protocolos como SPF (Sender Policy Framework ou estrutura de políticas do remetente), DKIM (DomainKeys Identified Mail ou correspondência identificada por chaves de domínio) e DMARC (Domain-based Message Authentication, Reporting & Conformance ou autenticação, relatórios e conformidade de mensagens baseados em domínio). Esses protocolos estabelecem confiança nas comunicações por email, o que reduz as chances de ataques bem-sucedidos de phishing e spam.

Objetivos abordados no exame:

- 4.5 Dado um cenário, modifique os recursos da empresa para aumentar a segurança.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Protocolos de Comunicação e Acesso

Sigla	Funcionamento Técnico	Tipo de Criptografia / Credencial
HTTPS	Utiliza o handshake TLS para estabelecer um canal cifrado antes de enviar dados HTTP.	Certificados Digitais (X.509) emitidos por uma autoridade (CA) e chaves assimétricas.
SMTPS	Inicia uma sessão segura (geralmente via comando STARTTLS ou porta 465) usando TLS.	Certificados Digitais no servidor para garantir a identidade e cifrar o túnel.
IMAPS	Criptografa a sessão de leitura de e-mails do início ao fim via TLS na porta 993.	Certificados Digitais no servidor; autenticação do usuário por senha cifrada.
SFTP	Todo o tráfego (dados e comandos) flui dentro de um túnel SSH (Secure Shell) binário.	Chaves SSH (pública/privada) ou senhas dentro do túnel seguro. Não usa certificados X.509.
FTPS	É o FTP clássico que solicita uma negociação TLS/SSL explícita ou implícita.	Certificados Digitais (similar ao HTTPS).
LDAPS	Envelopa as consultas de diretório dentro de um túnel TLS na porta 636.	Certificados Digitais para validar o servidor de domínio.
DNSSEC	Adiciona assinaturas digitais aos registros DNS existentes para validar a origem.	Chaves Criptográficas (Pares de Chaves) para assinar os registros. Não cifra o dado, apenas autentica.

Autenticação de E-mail

Aqui a coisa fica interessante, pois não estamos falando de "túneis", mas de **provas de identidade**.

Sigla	Funcionamento Técnico	Tipo de Criptografia / Credencial
SPF	O servidor de destino verifica um registro TXT no DNS do remetente para ver se o IP é autorizado.	Sem criptografia direta. Baseia-se na confiança da arquitetura DNS e em endereços IP.

Sigla	Funcionamento Técnico	Tipo de Criptografia / Credencial
DKIM	O servidor de envio assina o cabeçalho/corpo do e-mail com uma chave privada. O destino valida com a pública.	Chaves Assimétricas. Usa hashing (como SHA-256) e assinaturas digitais.
DMARC	Uma política no DNS que utiliza os resultados do SPF e DKIM para decidir o destino da mensagem.	Lógica de Política. Baseia-se nos resultados criptográficos do DKIM e nas verificações do SPF.

Muitos dos protocolos usados hoje nas redes de computadores foram desenvolvidos há muitas décadas, quando a funcionalidade era primordial, a confiabilidade era presumida em vez de conquistada e a segurança da rede era um problema menor do que é hoje. Boa parte dos protocolos dessa era pioneira têm alternativas seguras ou podem ser configurados para incorporar recursos de segurança, enquanto outros devem ser simplesmente evitados.

Os protocolos inseguros, como HTTP e Telnet, transmitem dados em formato de texto não criptografado, o que significa que qualquer pessoa que acesse os pacotes de dados pode ler quaisquer dados interceptados em uma rede. Por outro lado, protocolos seguros, como HTTPS e SSH (como alternativas a HTTP e Telnet), usam criptografia para proteger os dados transmitidos e melhorar a segurança.

O uso de HTTPS é crucial para impedir que informações confidenciais do usuário, como credenciais de login e dados inseridos em campos de formulários, sejam roubadas ao usar páginas da Web. Os engenheiros de sistemas e de rede devem usar SSH em vez de Telnet ao se conectar a servidores e equipamentos para garantir que suas informações de login, dados e comandos sejam criptografados.

Infelizmente, os protocolos seguros são normalmente mais complexos de implementar, gerenciar e manter quando comparados aos seus equivalentes inseguros e, portanto, muitas vezes são evitados ou desativados. Por exemplo, HTTPS requer a obtenção de um certificado SSL/TLS válido de uma autoridade de certificação (CA). Depois de obter o certificado apropriado, ele deve ser instalado e configurado corretamente em um servidor, o que requer mais habilidade, tempo e planejamento do que simplesmente habilitar e usar HTTP.

Os protocolos seguros utilizam criptografia e descriptografia, o que exige o manuseio correto de chaves criptográficas, incluindo processos sobre como elas são criadas, armazenadas, distribuídas e revogadas. Além disso, depois de obter e configurar adequadamente o certificado, ele deve ser gerenciado de forma eficaz para garantir que permaneça confiável e não expire.

A solução de problemas com protocolos seguros é mais desafiadora em comparação com contrapartes inseguras, porque os administradores não podem inspecionar facilmente o conteúdo dos pacotes de dados ao solucionar problemas, e a configuração de software e sistemas operacionais seguros é mais complicada e propensa a configurações incorretas em comparação com configurações simples que usam protocolos inseguros.

Apesar dessas complexidades, os benefícios de segurança oferecidos pelos protocolos seguros superam significativamente os desafios. Todos os protocolos devem ser seguros, a menos que justificativas específicas legitimem o uso de inseguros.

Implementação de protocolos seguros

As organizações geralmente seguem processos formais ao selecionar protocolos seguros para garantir uma documentação abrangente e uma tomada de decisões bem informada. Esses processos incluem ponderar riscos, revisar políticas e avaliar os recursos de segurança de diferentes protocolos. As organizações também podem consultar especialistas técnicos ou fornecedores para obter recomendações. Os resultados desses processos são documentados, o que é útil para auditorias e revisões de conformidade. Além disso, esses resultados normalmente afetarão os parâmetros de referência e os sistemas de gerenciamento de configurações.

A seleção de protocolos, atribuição de portas, configuração de métodos de transporte e outras considerações de segurança exigem uma reflexão cuidadosa. A primeira etapa requer a avaliação do tipo de dados usado e seu nível de confidencialidade. As organizações devem selecionar protocolos seguros (como HTTPS, SSH e SFTP/FTPS) para transmitir dados confidenciais ou privados. A configuração de portas TCP depende do protocolo, pois as portas padrão estão associadas a protocolos específicos (HTTP geralmente usa a porta 80, HTTPS usa a porta 443). Embora as portas padrão dos protocolos possam ser alteradas, isso pode complicar a configuração e causar possíveis problemas de acessibilidade.

No entanto, muitos administradores optam por alterar as portas padrão e escolhem um método para obscurecê-las. O TCP (Transmission Control Protocol, protocolo de controle de transmissão) e o UDP (User Datagram Protocol, protocolo de datagrama de usuário) são dois métodos de transporte principais. O TCP é orientado para a conexão e oferece confiabilidade, ordenação e verificação de erros, tornando-o adequado para aplicativos que exigem altos níveis de confiabilidade. O UDP não tem conexão, o que o torna mais rápido que o TCP e mais adequado para aplicativos em tempo real, como streaming de vídeo, telefonia e jogos, onde a perda ocasional de pacotes é menos impactante. Ao selecionar protocolos seguros, os administradores e analistas devem considerar níveis de criptografia adequados, métodos de autenticação, firewalls existentes ou outros equipamentos de segurança, além de outros fatores que possam afetar a operação dos sistemas e softwares que pretendem proteger. Em última análise, a seleção de protocolos requer um equilíbrio ideal entre segurança, capacidade de manutenção, desempenho e custo.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Anotações extras

HTTP e Telnet era transmissão de texto pura, então os dados fluíam na rede sem proteção nenhuma. Conforme o tempo foi passando e a segurança das redes foram pedindo mais, eles foram substituídos pelos seus sucessores HTTPS e SSH que são conexões seguras e criptografadas. HTTPS usa o modo de certificação de validação da Entidade Certificadora para ver se o site é realmente seguro (CA); o SSH faz o acesso em máquinas remotas, mas com uma chave criptografada simétrica para criptografar todos os dados da sessão. Mais sobre as dúvidas do SSL junto do nome

TLS está em [[Se o SSL não é mais usado, porque diabos ele fica na sigla junto do TLS ? Tipo assim SSL-TLS ?]]. Basicamente ficou um nome comercial tão forte pelo certificado SSL ter se enraizado que o nome acabou ficando. Hoje em dia, ele está morto e enterrado já, usamos apenas TLS. A sigla SSL (Secure Socket Layer) ficou para trás e se alguém usa, está vulnerável. Nesse trecho eu fiz uma pergunta ao Gemini falando pra ele me dar um exemplo de aplicativo TCP e UDP então crie a nota [[TCP vs UDP - Confiança e Velocidade]]. Só pra simplificar de cara é que o TCP preza por integridade dos dados, que os bits deles não sejam mudados ou perdidos no meio do caminho. UDP preza por velocidade e entrega dos dados, não importa de um deles caiu lá trás, ele quer que você receba.

Tal como acontece com outros protocolos de aplicação TCP/IP iniciais, as comunicações HTTP não são seguras. A camada de soquetes seguros (Secure Sockets Layer, SSL) foi desenvolvida pela Netscape na década de 1990 para resolver a falta de segurança em HTTP. O SSL teve grande popularidade no setor e foi rapidamente adotado como um padrão com o nome de [[Transport Layer Security (TLS)]]. Ela é, normalmente, usada com o aplicativo HTTP (chamado de HTTPS ou HTTP seguro), mas também pode ser usada para proteger outros protocolos de aplicativo e como uma solução de rede privada virtual (VPN).

Para implementar o TLS, um servidor recebe um certificado digital assinado por uma autoridade de certificação (Certificate Authority, CA) confiável. O certificado comprova a identidade do servidor (supondo que o cliente confie na autoridade de certificação) e valida o par de chaves pública/privada do servidor. O servidor usa seu par de chaves e o protocolo TLS para entrar em acordo com o cliente sobre cifras mutuamente suportadas e negociar uma sessão de comunicações criptografadas.

Nota:

O HTTPS opera na porta 443 por padrão. A operação do HTTPS é indicada pelo uso de https:// para o URL e por um ícone de cadeado mostrado no navegador.

É possível também instalar um certificado no cliente para que o servidor possa confiar nele. Isso não é usado com frequência na Web, mas é um recurso de VPNs e redes corporativas que exigem autenticação mútua.

Versões de SSL/TLS

Embora a sigla SSL ainda seja usada, as versões de segurança da camada de transporte (TLS) são as únicas seguras de usar. Um servidor pode fornecer suporte para clientes legados, mas isso é menos seguro, obviamente. Por exemplo, um servidor TLS 1.2 pode ser configurado para permitir que os clientes façam downgrade para TLS 1.1 ou 1.0 ou mesmo SSL 3.0 se não suportarem TLS 1.2.

Nota:

Um ataque de downgrade é quando um [[ataque on-path]] tenta forçar o uso de um conjunto de cifras e versão SSL/TLS fracas.

A versão 1.3 do TLS foi aprovada em 2018. Um dos principais recursos do TLS 1.3 é a remoção da capacidade de executar ataques de downgrade, impedindo o uso de recursos e algoritmos não seguros de versões anteriores. Também há alterações no protocolo de handshake para reduzir o número de mensagens e acelerar as conexões.

Conjuntos de cifras

Um [[conjunto de cifras (cipher suite)]] é o conjunto de algoritmos suportados tanto pelo cliente quanto pelo servidor para realizar as diferentes operações de criptografia e hash exigidas pelo protocolo. Antes do TLS 1.3, um conjunto de cifras seria escrito da seguinte forma:

```
ECDHE-RSA-AES128-GCM-SHA256
```

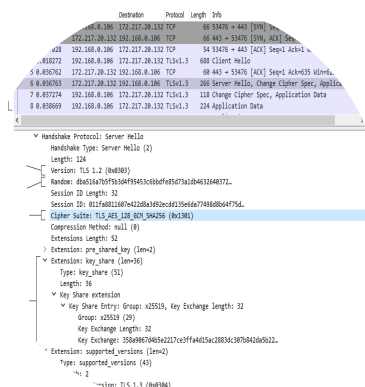
Isso significa que o servidor pode usar o modo efêmero Diffie-Hellman sobre curvas elípticas para acordo de chave de sessão, assinaturas RSA, AES-GCM de 128 bits (Galois Counter Mode) para criptografia em massa simétrica e SHA de 256 bits para [[funções HMAC]]. Os conjuntos que o servidor prefere são listados primeiro em sua lista de cifras suportadas.

O TLS 1.3 usa conjuntos simplificados e encurtados. Um conjunto de cifras TLS 1.3 típico aparece da seguinte maneira:

```
TLS_AES_256_GCM_SHA384
```

Apenas acordos de chave efêmera são suportados no 1.3 e o tipo de assinatura é fornecido no certificado, portanto, o conjunto de cifras lista apenas a força da chave de criptografia em massa e o modo de operação (AES_256_GCM), além do algoritmo de hash criptográfico (SHA384) usado no âmbito da nova função de derivação de chave hash (Hash Key Derivation Function, HKDF). O HKDF é o mecanismo pelo qual o segredo compartilhado estabelecido pelo acordo de chave D-H é usado para derivar chaves de sessão simétricas.

Figura 1. Visualização do handshake TLS em uma captura de pacotes no Wireshark. Observe que a conexão está usando TLS 1.3 e um dos conjuntos de cifras encurtado (TLS_AES_128_GCM_SHA256).





This content is only available on larger screen sizes. Please revisit this page on a larger device.

Anotações extras

Em pesquisa, TLS nada mais é do que a criação de um túnel seguro para a comunicação de dispositivos na internet. Sem o TLS, as nossas informações viajam pela rede por texto simples. Com TLS, os dados são criptografados, onde eles criam o túnel por chaves assimétricas que é um processo mais lento e depois do túnel criado, eles trocam chaves simétricas para maior velocidade. Consulte mais na nota em [\[\[Exemplo Prático de TLS em Rede\]\]](#). Mais sobre esse conteúdo resumido pelo Gemini está em [\[\[Me explique de forma resumida esse capítulo, foi difícil de entender a última parte\]\]](#). Eu fiz algumas perguntas pertinentes a geração de certificados também e CAs no geral. Também existe uma nota falando sobre o DH que está em [\[\[O que é o Diffie-Hellman em cibersegurança ? Me explique o que é de forma simples-objetiva. Também quero que me dê um exemplo simples dele para que eu possa fixar.\]\]](#) Chaves efêmeras são só um sinônimo para dizer descartável, ou seja, se um invasor tem sua chave de acesso da sessão em uma ocasião e a chave não for efêmera, diga adeus a sua privacidade e dados, porque todas as portas que você fechou atrás de você a 2 anos atrás, ele terá acesso. Já com a chave efêmera é como se eu e você tivéssemos combinado uma palavra chave para trocar informações e trocamos elas a cada conversa. Mais sobre esse trecho final em [\[\[TLS 1.3 Simplificando Segurança na Web\]\]](#)

Um diretório de rede lista os sujeitos (principalmente usuários, computadores e serviços) e objetos (como diretórios e arquivos) disponíveis na rede, além das permissões que os sujeitos têm sobre os objetos. Um diretório facilita a autenticação e a autorização, e é fundamental que ele seja mantido como um serviço altamente seguro. A maioria dos serviços de diretório é baseada no Lightweight Directory Access Protocol (LDAP), executado na porta 389. O protocolo básico não fornece segurança e todas as transmissões são em texto simples, tornando-o vulnerável a ataques de detecção e ataques on-path. A autenticação, conhecida como associação ao servidor, pode ser implementada das seguintes maneiras:

- **Sem autenticação:** é concedido acesso anônimo ao diretório.
- **Simple Bind (associação simples):** o cliente deve fornecer seu nome diferenciado (Distinguished Name, DN) e senha, que são passados como texto simples.
- **Simple Authentication and Security Layer, SASL (autenticação simples e camada de segurança):** o cliente e o servidor negociam o uso de um mecanismo de autenticação suportado por ambos, como Kerberos. O comando STARTTLS pode ser usado para exigir criptografia (validação) e integridade da mensagem (assinatura). Este é o mecanismo preferencial para a implementação do LDAP pelo Active Directory (AD) da Microsoft.

- [[LDAP Secure (LDAPS)]]: o servidor é instalado com um certificado digital, que é usado para configurar um túnel seguro para a troca de credenciais do usuário. O LDAPS usa a porta 636.

Se for necessário acesso seguro, os métodos de acesso por autenticação simples e anônima devem ser desativados no servidor.

Geralmente, dois níveis de acesso devem ser concedidos no diretório: acesso somente para leitura (consulta) e acesso para leitura/gravação (atualização). Isto é implementado usando uma política de controle de acesso, mas o mecanismo exato é específico ao fornecedor e não é discriminado na documentação dos padrões de LDAP.

A menos que hospede um serviço público, o servidor de diretório LDAP deve ser acessível apenas a partir da rede privada. Ou seja, a porta LDAP deve ter o acesso através da interface pública bloqueado por um firewall. Se houver integração com outros serviços pela Internet, o ideal é que sejam permitidos apenas IPs autorizados.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Anotações extras

Aqui sem muitos comentários. LDAP nada mais é do que uma lista gigantesca na rede nos dizendo usuários, máquinas e diretórios dentro da rede. Ele trafega em texto simples, mas também pode ser configurado para ser mais seguro usando o LDAPS que o servidor é instalado com certificado para criar um túnel seguro para troca de credenciais de usuários, que fica na porta 636. A microsoft recomenda usar o método de autenticação com STARTTLS com a integração via AD.

O [[Simple Network Management Protocol (SNMP)]] é uma estrutura amplamente utilizada para gerenciamento e monitoramento. O SNMP consiste em um monitor SNMP e agentes.

- O agente é um processo (software ou firmware) em execução em um switch, roteador, servidor ou outro dispositivo de rede compatível com SNMP.
- Esse agente mantém um banco de dados chamado de base de informação de gerenciamento (Management Information Base, MIB), que mantém estatísticas em relação à atividade do dispositivo. Um exemplo dessa estatística é o número de quadros por segundo manipulado por um switch. O agente também é capaz de iniciar uma operação de interceptação na qual informa ao sistema de gerenciamento sobre um evento importante (falha da porta, por exemplo). O limite para o acionamento de interceptações pode ser definido para cada valor. Consultas do dispositivo ocorrem pela porta 161 (UDP); as interceptações são comunicadas pela porta 162 (também UDP).

- O monitor SNMP (um programa de software) fornece um local do qual a atividade da rede pode ser supervisionada. Ele monitora todos os agentes analisando-os a intervalos regulares para obter informações de seus MIBs e exibe as informações para revisão. Ele também exibe todas as operações de interceptação como alertas para o administrador da rede avaliar e tomar as medidas necessárias.

Se o SNMP não for usado, ele deveria ser desativado. Ao executar o SNMP, algumas diretrizes importantes são:

- Os nomes da comunidade SNMP são enviados em texto simples e, portanto, não devem ser transmitidos pela rede se houver algum risco de serem interceptados.
- Use nomes de comunidade difíceis de adivinhar; nunca deixe o nome da comunidade em branco ou definido como o padrão.
- Use listas de controle de acesso para restringir as operações de gerenciamento a hosts conhecidos (ou seja, restringir a um ou dois endereços IP de hosts).
- Use o SNMP v3 sempre que possível e desative as versões mais antigas do SNMP. O SNMP v3 é compatível com criptografia e autenticação forte originada pelo usuário. Em vez de nomes de comunidades, os agentes são configurados com uma lista de nomes de usuários e permissões de acesso. Quando a autenticação é necessária, as mensagens SNMP são assinadas com um hash da senha do usuário. O agente pode verificar a assinatura e autenticar o usuário usando seu próprio registro da senha.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Eu fiz uma pergunta ao Gemini em [[Zabbix e protocolo SNMP - Monitoramento Essencial]] e recebi um retorno positivo do que eu já suspeitava. o SNMP seria aquele tira disfarçado entre os outros, o agente infiltrado, o informante do que acontece na para de o policial perguntar o que é que tá pegando. Peguei o exemplo do Zabbix que usa muito o SNMP para perguntar através dele o que está acontecendo com tal equipamento. E também não adianta usar versões antigas com v1 ou v2 onde ele mostras todas as informações em textos simples. Ou seja, se alguém tá bisbilhotando a rede com wireshark, consegue ver tudo. Os MIBs seriam os bancos de dados que eles criam para consultar as informações que lhes são solicitadas.

Existem muitas maneiras de transferir arquivos entre redes. Um sistema operacional de rede pode hospedar pastas e arquivos compartilhados, permitindo que eles sejam copiados ou acessados pela rede local ou via acesso remoto (por meio de uma VPN, por exemplo). Aplicativos de email e mensagens podem enviar arquivos como anexos. O HTTP é compatível com o download de arquivos (e uploads por meio de vários mecanismos de script). Há também serviços de compartilhamento de arquivos ponto a ponto.

Apesar de estarem disponíveis protocolos e serviços mais recentes, o protocolo de transferência de arquivos (File Transfer Protocol, FTP) continua muito popular porque é eficiente e tem amplo suporte a várias plataformas.

Protocolo de transferência de arquivo

Um servidor [[File Transfer Protocol (FTP)]] normalmente é configurado com vários diretórios públicos, hospedando arquivos e contas de usuários. A maioria dos servidores HTTP também funciona como servidor FTP, e serviços, contas e diretórios FTP podem ser instalados e habilitados por padrão ao instalar um servidor Web. O FTP é mais eficiente do que anexos de arquivo ou transferência de arquivos por HTTP, mas não tem mecanismos de segurança. Toda a autenticação e transferência de dados são comunicadas como texto simples, o que significa que as credenciais são facilmente identificadas entre o tráfego FTP interceptado.

Nota:

Você deve assegurar-se de que os usuários não instalem servidores não autorizados em seus PCs (servidores maliciosos). Por exemplo, é disponibilizada com versões de cliente do Windows uma versão do IIS que inclui servidores HTTP, FTP e SMTP, embora não seja instalada por padrão.

SSH FTP (SFTP) e FTP sobre SSL (FTPS)

O [[Secure FTP (SFTP)]] resolve os problemas de privacidade e integridade do FTP criptografando a autenticação e a transferência de dados entre cliente e servidor. No SFTP, um link seguro é criado entre o cliente e o servidor usando o Secure Shell (SSH) pela porta TCP 22. Assim, comandos FTP comuns e transferências de dados podem ser enviados pelo link seguro sem o risco de espionagem ou ataques on-path. Essa solução requer um servidor SSH compatível com SFTP e o software cliente SFTP.

Outro meio de proteger o FTP é usar o protocolo de segurança da conexão SSL/TLS. Há dois modos de fazer isso:

- **TLS explícito (FTPES):** utiliza o comando AUTH TLS para atualizar uma conexão não segura estabelecida pela porta 21 para uma conexão segura. Isso protege as credenciais de autenticação. A conexão de dados das transferências de arquivos também pode ser criptografada (usando o comando PROT).
- **[[TLS implícito (FTPS)]]:** negocie um túnel SSL/TLS antes da troca de comandos do FTP. Este modo usa a porta segura 990 para a conexão de controle.

O FTPS é complicado de configurar quando há firewalls entre o cliente e o servidor. Consequentemente, o FTPES costuma ser o método preferido.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Não tem como falar de transferência de arquivos sem falar do protocolo FTP. E pra melhorar, o protocolo HTTP tem suporte a vários tipos de mecanismos de downloads e uploads de arquivos. Um deles você já sabe qual é. o FTP ainda continua sendo a opção mais popular ainda por sua facilidade e suporte disponível a ele, por mais que tenhamos outras opções ainda. A maioria dos servidores HTTP funcionam como servidores FTP. Existem duas derivações dele mais seguras e que são recomendadas hoje em dia que são a SFTP que seria a versão de transferência de arquivos via link seguro e a FTPS que negocia túnel SSL/TLS antes da troca de arquivos. Ambos resolvem problemas de privacidade da versão original dele.

Os serviços de email usam dois tipos de protocolos:

- O [[Simple Mail Transfer Protocol (SMTP)]] especifica como o correio eletrônico é enviado de um sistema para outro.
- Um protocolo de caixa de correio armazena mensagens para os usuários e permite que eles as baixem para computadores clientes ou as gerenciem no servidor.

Secure SMTP (SMTPS)

Para entregar uma mensagem, o servidor SMTP do remetente descobre o endereço IP do servidor SMTP do destinatário com base no nome de domínio contido no endereço de email. O servidor SMTP do domínio é registrado no DNS usando um registro MX (Mail Exchanger).

As comunicações SMTP podem ser protegidas usando TLS. O funcionamento é muito parecido com HTTPS com um certificado no servidor SMTP. O SMTP usa a TLS de duas maneiras:

- **STARTTLS**: é um comando que atualiza uma conexão não segura existente para utilizar TLS. Também é denominado TLS explícito ou TLS oportunista. Oportunista porque vê uma brecha e tenta "convencer" a usar o TLS.
- **SMTPS**: estabelece uma conexão segura antes que quaisquer comandos SMTP (HELO, por exemplo) sejam trocados. Ele também é denominado TLS implícito. Impõe a conexão segura antes mesmo que ela comece, diferente do STARTTLS, ele exige.

O método STARTTLS é mais amplamente implementado do que SMTPS. As configurações típicas de SMTP usam as seguintes portas e serviços seguros:

- **Porta 25:** é usada para retransmissão de mensagem (entre servidores SMTP ou agentes de transferência de mensagem [Message Transfer Agents, MTA]). Se a segurança for necessária e permitida pelos dois servidores, o comando STARTTLS pode ser usado para configurar a conexão segura.
- **Porta 587:** é usada por clientes de email (agentes de envio de mensagens [Message Transfer Agents, MTA]) para enviar mensagens para entrega por um servidor SMTP. Os servidores configurados para aceitar a porta 587 devem usar STARTTLS e exigir autenticação antes do envio da mensagem.
- **Porta 465:** é usada por alguns provedores e clientes de email para envio de mensagens por TLS implícito (SMTPS), embora esse uso agora seja obsoleto pela documentação padronizadora.

Secure POP (POP3S)

O [[Post Office Protocol v3 (POP3)]] é um protocolo de caixa postal projetado para armazenar, em um servidor, as mensagens entregues pelo SMTP. Quando o cliente se conecta à caixa de correio, o POP3 baixa as mensagens para o cliente de email do destinatário.

Figura 1. Configuração de protocolos de acesso à caixa de correio em um servidor



Um aplicativo cliente que usa POP3, como Microsoft Outlook ou Mozilla Thunderbird, estabelece uma conexão TCP com o servidor POP3 pela porta 110. O usuário é autenticado (com nome de usuário e senha) e o conteúdo da caixa de correio é baixado para processamento no computador local. O POP3S é a versão segura do protocolo que opera na porta TCP 995 por padrão.

Secure IMAP (IMAPS)

Em comparação ao POP3, o [[Internet Message Access Protocol (IMAP)]] oferece suporte a conexões permanentes com o servidor e permite que vários clientes se conectem simultaneamente à mesma caixa postal. Ele também permite que um cliente administre pastas de correspondência eletrônica no servidor. Os clientes se conectam ao IMAP pela porta TCP 143. Eles se autenticam, e a seguir acessam mensagens nas pastas designadas. Como em outros protocolos de email, a conexão pode ser protegida estabelecendo um túnel SSL/TLS. A porta padrão para IMAPS é a TCP 993.

1. DKIM (A Lacre de Cera Personalizado) ☐

O DKIM é como se você, antes de enviar a garrafa, colocasse um lacre de cera com o brasão da sua família no gargalo.

- O segredo: Só você tem o anel para fazer esse lacre (sua chave privada).
- A conferência: No verso da garrafa (ou no DNS, para os nerds), tem uma instrução dizendo: "Para verificar se este lacre é meu, use este molde público aqui".
- **A utilidade:** Se alguém abrir a garrafa no caminho para beber um gole e colocar água (mudar o conteúdo do e-mail), o lacre quebra. Se o lacre estiver violado ou for falso, o destinatário sabe que a mensagem foi mexida.

[[Domain-based Message Authentication, Reporting & Conformance (DMARC)]] utiliza os resultados das verificações SPF e DKIM para definir regras sobre o tratamento das mensagens, como mover mensagens para quarentena ou spam, rejeitá-las diretamente ou sinalizá-las. O DMARC também fornece recursos de relatório, dando ao proprietário de um domínio visibilidade sobre quais sistemas estão enviando emails em seu nome, incluindo quaisquer atividades não autorizadas.

Sem o blá blá blá todo - parte 2

2. DMARC (O Manual de Instruções do Segurança) ☐

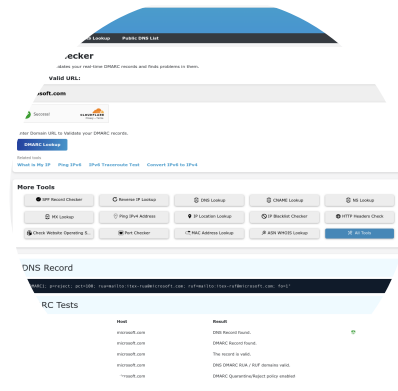
O DMARC é o "chefe da segurança" que fica na guarita do destinatário. Ele não faz o trabalho braçal, ele dá as ordens baseadas no que o DKIM e o SPF encontraram.

- Ele basicamente diz: "Se a carta chegar sem o lacre (DKIM) ou se o entregador não estiver na lista autorizada (SPF), faça o seguinte: **1. Ignore, 2. Jogue no lixo (Quarentena)** ou **3. Queime a carta e nem me avise (Reject)**".
- Além disso, ele anota tudo num caderninho (Relatórios) e te manda uma mensagem depois dizendo: "Olha, tentaram entregar 50 garrafas falsas com o seu nome hoje. Só pra você saber".

Consulte a nota que criei para facilitar a nossa vida. Passe o mouse em cima de [[SPF, DKIM e DMARC]]

Sem isso, qualquer um pode enviar um e-mail em seu nome dizendo para o seu financeiro pagar um boleto falso. E se você não configurar o DMARC direito, você está basicamente deixando a porta da frente aberta com um tapete de "Bem-vindo" para golpistas.

Figura 2. Realizando uma consulta DMARC no site DNSChecker <https://dnschecker.org> (<https://dnschecker.org>).



O uso combinado de SPF, DKIM e DMARC aumenta significativamente a segurança dos emails, tornando muito mais difícil para os invasores se passarem por domínios confiáveis, o que é uma das táticas mais comuns usadas em ataques de phishing e spam. Esses protocolos são ferramentas fundamentais no combate às ameaças por email, pois oferecem mecanismos essenciais que ajudam a verificar a autenticidade dos emails, manter a integridade do seu conteúdo e garantir a entrega segura da comunicação eletrônica.

Gateway de email

Um gateway (porta de entrada) de email é o ponto de controle para todo o tráfego de entrada e saída de emails. Ele atua como um guardião, examinando todos os emails para remover possíveis ameaças antes que elas cheguem às caixas de entrada. Os gateways de email utilizam várias medidas de segurança, incluindo filtros anti-spam, scanners antivírus e sofisticados algoritmos de detecção de ameaças para identificar tentativas de phishing, URLs mal-intencionados e anexos prejudiciais. Os gateways de email utilizam DMARC, SPF e DKIM para automatizar a autenticação e a validação dos remetentes de email, reduzindo as chances de que emails falsificados sejam entregues.

Os gateways de email também desempenham um papel crítico na aplicação e cumprimento de políticas, permitindo que as organizações criem regras relacionadas ao conteúdo e aos anexos de emails com base em políticas estabelecidas ou requisitos de conformidade regulamentar. O bloqueio de anexos, a filtragem de conteúdo e a prevenção contra perda de dados são tarefas comuns com as quais os gateways de email devem lidar.

O gateway de e-mail é quem faz o pente fino de tudo que chega do conjunto da santa trindade (SPF, DKIM e DMARC). Ele inspeciona todos os pacotes antes de entrega-lo a pessoa. Uma analogia seria na parte de uma cozinha de praia que falei com o Gemini sobre em [[Gateway de e-mail]]

Secure/Multipurpose Internet Mail Extensions

Secure/Multipurpose Internet Mail Extensions (S/MIME), ou extensões seguras/multifunções para mensagens de Internet, é um protocolo para proteger as comunicações por email. Ele criptografa emails e permite a autenticação do remetente para garantir a confidencialidade e a integridade das comunicações por email. O S/MIME usa técnicas de criptografia de chave pública para proteger o conteúdo do email (o “corpo” do email). Ele também incorpora assinaturas

digitais para apoiar a verificação do remetente e garantir que as mensagens não sejam modificadas. Ao fornecer recursos de criptografia e autenticação, o S/MIME aumenta significativamente a segurança da comunicação por email, mas sua implementação é geralmente complicada e propensa a configurações incorretas.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

O email é um dos canais de comunicação mais usados nas organizações. O e-mail serve como canal para dados confidenciais, como informações financeiras, propriedade intelectual, dados de clientes e funcionários, além de informações pessoais identificáveis (PII). Dada sua popularidade e a natureza sensível dos dados frequentemente trafegados, o e-mail é um vetor comum de perda de dados, tornando as proteções de [[prevenção contra perda de dados (DLP)]] extremamente importantes.

Embora conveniente, a facilidade de uso e os recursos de transmissão rápida do email podem, inadvertidamente, incentivar o manuseio descuidado de dados confidenciais. Erros humanos, como enviar dados confidenciais para os destinatários errados ou não usar métodos seguros para a transmissão de dados, são comuns e ressaltam a importância das medidas de DLP. Além disso, as soluções de DLP são cruciais para se proteger contra ameaças internas. Agentes internos podem representar riscos significativos de vazamento de dados devido à falta de conhecimento das políticas ou às más intenções.

Regulamentos como GDPR, HIPAA e PCI DSS impõem requisitos rigorosos para proteger tipos de dados específicos, com a DLP servindo como um mecanismo fundamental para garantir a conformidade e evitar a transmissão não autorizada de dados. Além disso, como o email é um dos alvos principais de ataques, as proteções DLP reduzem significativamente o risco de perda de dados, garantindo o manuseio e a proteção adequados de informações confidenciais.

As tecnologias de prevenção contra perda de dados (Data Loss Prevention,DLP) impedem o compartilhamento ou a divulgação não autorizada de informações confidenciais. As políticas de DLP são essenciais para monitorar e controlar o conteúdo usado em plataformas de comunicação, como o email. A DLP verifica emails e anexos em busca de certos tipos de informações confidenciais definidas pelas políticas de DLP da organização, incluindo números de cartão de crédito, números de previdência social, informações privadas ou quaisquer dados confidenciais. Se um email contiver esses tipos de informações, o sistema DLP pode executar várias ações com base em regras predefinidas, como bloquear o email, alertar o remetente ou o administrador ou criptografá-lo automaticamente antes da transmissão.

Aplicar a DLP nas comunicações por email é essencial para muitas organizações, especialmente aquelas que lidam com dados confidenciais de clientes ou estão sujeitas a regulamentos como GDPR, HIPAA ou PCI DSS. A DLP ajuda as organizações a minimizar o risco de violações de dados, evitar penalidades por inconformidade e manter a segurança e

a privacidade dos dados. A DLP é frequentemente aplicada usando gateways de email e políticas de segurança em ferramentas de proteção de endpoint.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Ele trata de proteção contra vazamento de dados e erros humanos, sem eles intencionais ou não. o DLP é usado em grande parte junto com o gateway de e-mail por motivos óbvios de verificação de anexos e conteúdos nos e-mails. Usar o DLP minimiza os riscos de uma empresa que a maior fonte de comunicação e transição de dados sensíveis é o e-mail.

A filtragem de Domain Name System (DNS) (sistema de nomes de domínio) é uma técnica que bloqueia ou permite o acesso a sites específicos controlando a resolução de nomes de domínio em endereços IP. Ela opera com base no princípio de que, para que um dispositivo acesse um site, ele deve primeiro resolver o nome de domínio deste no respectivo endereço IP associado, um processo gerenciado pelo DNS. Quando é feita uma solicitação para resolver o URI de um site, o filtro DNS verifica a solicitação em um banco de dados de nomes de domínio. Se o domínio estiver associado a atividades mal-intencionadas ou estiver em uma lista não aprovada por algum motivo, o filtro bloqueará a solicitação, impedindo o acesso ao site potencialmente prejudicial.

A filtragem de DNS é altamente eficaz por vários motivos. Veja a seguir:

- Fornece um mecanismo de defesa proativo, que bloqueia o acesso a sites de phishing conhecidos, sites de distribuição de malware e outros destinos on-line mal-intencionados.
- Pode ajudar a aplicar as políticas de uso aceitável (AUPs) de uma organização com bloqueando o acesso a sites inadequados ou que causem distrações e garantindo que a Internet seja usada de forma responsável e produtiva.
- Pode proteger todos os dispositivos conectados a uma rede, incluindo dispositivos de IoT, ao fornecer uma camada extra de segurança.
- É uma solução simples que é fácil de implementar e apresenta risco mínimo, o que a torna um controle de segurança econômico adequado para redes de qualquer porte.

Embora a filtragem de DNS seja altamente eficaz, ela deve ser combinada com outras medidas de segurança para oferecer uma proteção abrangente. Resumindo, filtragem de DNS é uma maravilha, fácil, prático e super eficaz para se ter em uma rede onde o foco é ter segurança.

consulta Gemini

Vamos direto ao ponto: se você acha que a segurança da sua rede depende apenas de um firewall caro, você está redondamente enganado. A filtragem de DNS é aquele herói silencioso que faz o trabalho sujo antes mesmo do problema bater à sua porta.

Imagine que o **DNS** é o GPS do seu carro. Você não sabe onde fica a "Rua dos Bobos, número 0", mas o GPS sabe as coordenadas latitudinais e longitudinais (o endereço IP). Navegar na internet sem DNS seria como tentar decorar o CPF de cada pessoa que você quer ligar. Impossível.

A Analogia do GPS com "Filtro Anti-Cilada" ☐☐

A filtragem de DNS funciona como se o seu GPS tivesse um sensor de "vizinhança perigosa". Quando você digita um site, o filtro faz o seguinte:

1. **A Consulta:** Você pede para ir ao site `banco-falso-muito-confiavel.com`.
2. **A Checagem:** Antes de te dar o endereço, o filtro consulta uma "lista negra" de endereços conhecidos por serem esconderijos de bandidos (sites de phishing ou malware).
3. **O Bloqueio:** Se o site estiver na lista, o filtro simplesmente diz: "Não conheço esse caminho" ou te manda para uma página de aviso. Você nem chega a ver o site malicioso.

Por que isso é genial (e prático)? ☐✂

- **Defesa Proativa:** Ele corta o mal pela raiz. Bloqueia o acesso a sites de phishing conhecidos e destinos mal-intencionados antes que qualquer código maligno tente entrar na sua máquina.
- **Babá de IoT (Internet of Trash):** Dispositivos inteligentes (como aquela sua torradeira Wi-Fi ridícula) são famosamente burros em termos de segurança. Eles não rodam antivírus. Mas todos eles precisam de DNS. Se a torradeira tentar se comunicar com um servidor de controle na Rússia, o filtro de DNS corta a conversa na hora.
- **Foco no Trabalho (AUP):** Quer impedir que seus funcionários passem o dia vendo vídeos de gatinhos ou em sites de apostas? É só bloquear essas categorias no DNS. É mais barato que contratar um supervisor e muito mais eficiente.

A grande beleza aqui é a simplicidade: você muda os servidores de DNS na configuração do roteador e, *pá*, toda a sua rede ganha uma camada extra de proteção sem que ninguém precise instalar nada em cada celular ou computador.

Implementação da filtragem de DNS

A filtragem de DNS é implementada usando diferentes métodos e ferramentas. A prevalent method is through DNS filtering services like Cisco's OpenDNS (<https://www.opendns.com/> (<https://www.opendns.com/>)), Quad9 (<https://www.quad9.net/> (<https://www.quad9.net/>)), or CleanBrowsing (<https://cleanbrowsing.org/> (<https://cleanbrowsing.org/>)). Esses serviços fornecem resolução de DNS com filtragem integrada, exigindo apenas que organizações e usuários redirecionem suas solicitações de DNS para os servidores de DNS do serviço de filtragem. Serviços de DNS que já tem filtragem de conteúdo. Alguns deles que dei olhada bloqueiam sites adultos e coisas violetas. Seria ideia para colocar na rede de uma família.

As organizações que gerenciam seus próprios servidores de DNS, como o servidor de DNS da Microsoft ou o BIND, podem implementar diretamente a filtragem de DNS. Esse método, embora mais complexo, fornece controle completo sobre as políticas de filtragem e permite a integração de listas de blocos ou [[feeds de zona de política de resposta (Response Policy Zone, RPZ)]] nas configurações do servidor.

Outra estratégia envolve o uso de firewalls de DNS, que interceptam consultas de DNS na rede e aplicam regras de filtragem de acordo. Algumas ferramentas de proteção de endpoints e software antivírus oferecem recursos de filtragem de DNS para a proteção dos dispositivos em si, o que é ideal para notebooks e outros dispositivos móveis que podem se conectar a várias redes com diferentes níveis de segurança ativados por padrão.

O software de código aberto Pi-hole (<https://pi-hole.net/>) ou ADGuard (<https://github.com/AdguardTeam/AdguardHome>) pode ser configurado como um resolvidor de DNS local com recursos de filtragem. Este software é executado no Linux e comumente implementado usando o hardware Raspberry Pi devido à sua sobrecarga de baixo desempenho. Independentemente do método escolhido, a personalização das políticas de filtragem permite categorizar sites para simplificar a criação de listas de blocos ou permitir listas por requisitos. Manter os filtros de DNS atualizados é essencial para uma filtragem eficaz, que acompanhe as ameaças em evolução e as necessidades organizacionais em constante mudança.

Figura 1. Painel administrativo do Pi-hole mostrando estatísticas de resolução de DNS.



Captura de tela cortesia do Pi-hole.

Descrição

O status é exibido no canto superior esquerdo. A opção Dashboard está selecionada no painel à esquerda. O dashboard exibe: total de consultas (5 clientes): 3.416; consultas bloqueadas: 491; percentual bloqueado: 14,4%; e domínios na lista de bloqueios: 82.309. Dois gráficos de barras verticais são intitulados total de consultas nas últimas 24 horas e atividade dos clientes nas últimas 24 horas.

Segurança de DNS

O DNS é um serviço crucial que deve ser configurado para tolerar falhas. É mais difícil lançar ataques DoS contra os servidores que realizam a resolução de nomes da Internet, mas, se um invasor conseguir atingir o servidor DNS em uma rede privada, é possível interromper seriamente a operação dela.

Os ataques ao DNS também podem atingir o aplicativo e/ou a configuração do servidor. Muitos serviços de DNS são executados no BIND (Berkley Internet Name Domain), distribuído pelo Internet Systems Consortium ([isc.org](https://www.isc.org/) (<https://www.isc.org/>)). Existem vulnerabilidades conhecidas em muitas versões do servidor BIND, por isso é fundamental atualizar o servidor para a versão mais recente. O mesmo conselho geral se aplica a outros softwares de servidor DNS, como o da Microsoft. Obtenha e verifique anúncios de segurança e, em seguida, teste e aplique correções e atualizações críticas ou relacionadas à segurança.

As [Extensões de Segurança DNS (DNSSEC)] ajudam a mitigar ataques de spoofing e envenenamento ao fornecer um processo de validação para as respostas DNS. Com o DNSSEC ativado, o servidor de referência para a zona cria um “pacote” de registros de recursos (chamado de RRset) assinado com uma chave privada (chave de assinatura da zona, ou Zone Signing Key). Quando outro servidor solicita uma troca segura de registros, o servidor autoritativo devolve o pacote junto da sua chave pública, que pode ser usada para verificar a assinatura.

Figura 2. Serviços de DNS do servidor Windows com DNSSEC ativado

[illegible]

Descrição

As abas na parte superior são: arquivo, ação, exibir e ajuda. A opção classroom dot local sob a pasta de zonas de pesquisa direta (forward lookup zones) está selecionada no painel à esquerda. O painel à direita exibe uma tabela que lista o nome, o tipo, os dados e o carimbo de data/hora.

A chave de assinatura de chaves aplicada a um domínio específico é validada pelo domínio principal ou ISP do host. As autoridades dos domínios de nível superior são validadas pelos Registros Regionais da Internet e os servidores root de DNS são autovalidados, usando um tipo de assinatura de chave de grupo de controle M-of-N. Isso estabelece uma cadeia de confiança desde os servidores root até qualquer subdomínio específico.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

A segurança de aplicativos para web e nuvem inclui o hardening da nuvem, que fortalece a infraestrutura da nuvem e reduz sua superfície de ataque, e a segurança de aplicativos, que garante que o software seja criado, desenvolvido e implantado com segurança. Ambas as práticas operam juntas para estabelecer uma estratégia de defesa em camadas, o que protege de forma eficaz contra diversas ameaças diferentes. As práticas de codificação seguras incluem técnicas de validação de entrada, incorporando o princípio do privilégio mínimo, mantendo o gerenciamento seguro das sessões, aplicando criptografia, suporte a patches e muitos outros recursos. Além disso, os desenvolvedores devem criar softwares que produzam logs abrangentes, estruturados e significativos, incorporando mecanismos de alerta em tempo real. Essas práticas complementares proporcionam um ambiente seguro e protegido para os aplicativos web e de nuvem.

Objetivos abordados no exame:

- 4.1 Considerando determinado cenário, aplicar técnicas de segurança comuns aos recursos de computação.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

As considerações de segurança para novas tecnologias de programação devem ser bem compreendidas e testadas antes da implantação. Um dos desafios do desenvolvimento de aplicativos é a enorme pressão para liberar uma solução, que muitas vezes supera qualquer exigência de garantir que o aplicativo seja seguro. Um processo de design de software legado pode estar fortemente focado em elementos de alta visibilidade, como funcionalidade, desempenho e custos. As práticas de desenvolvimento modernas usam um ciclo de vida de desenvolvimento de segurança executado em paralelo ou integrado com o foco na funcionalidade e usabilidade do software. Alguns exemplos são o SDL da Microsoft ([microsoft.com/en-us/securityengineering/sdl](https://www.microsoft.com/en-us/securityengineering/sdl)) e o Modelo de Maturidade do Software Assurance da OWASP (owasp.org/www-project-samm/) e o Security Knowledge Framework (<https://owasp.org/projects/spotlight/historical/2021.02.03/>). A OWASP também reúne descrições de vulnerabilidades, explorações e técnicas de mitigação específicas, como o Top 10 da OWASP (owasp.org/www-project-top-ten/).

Validação de entrada

A [validação de entrada] é uma técnica essencial de proteção utilizada no desenvolvimento de softwares e aplicações web, que aborda o problema de entradas não confiáveis. A entrada não confiável se refere à forma como um invasor pode fornecer dados especialmente criados para um aplicativo a fim de manipular seu comportamento. Os ataques de injeção exploram os mecanismos de entrada dos quais os aplicativos dependem para executar comandos e scripts mal-intencionados, com os quais procuram acessar dados confidenciais, controlar o funcionamento do aplicativo, obter acesso a sistemas de back-end protegidos e interromper operações.

Nota:

A OWASP oferece uma excelente visão geral sobre validação de entrada.

Sem uma validação de entrada eficaz, os aplicativos são vulneráveis a muitas classes diferentes de ataques de injeção, como injeção de SQL, injeção de código, cross-site scripting (XSS) e muitos outros.

Método de validação	Descrição
Lista de permissões	Este método permite apenas entradas que correspondam a um conjunto predeterminado e aprovado de valores ou padrões.
Lista de bloqueio	Esta estratégia bloqueia explicitamente entradas prejudiciais conhecidas, como certos caracteres especiais ou padrões comumente usados em ataques.
Verificações de tipo de dados	Estas verificações garantem que os dados de entrada sejam do tipo esperado, como uma string, número inteiro ou data.
Verificações de intervalo	Validam se as entradas numéricas estão dentro dos intervalos esperados.
Expressões regulares	Também conhecidas como regex, elas são usadas para combinar a entrada com padrões esperados ou sinais de atividade mal-intencionada.
Codificação	Ajuda a impedir com segurança e confiabilidade que caracteres especiais na entrada sejam interpretados como comandos ou scripts executáveis.

Cookies seguros

[[Cookies]] são pequenos fragmentos de dados armazenados em um computador por um navegador web durante o acesso a um site. Eles mantêm os estados da sessão, lembram as preferências do usuário e rastreiam o comportamento do usuário e outras configurações. Os cookies podem ficar vulneráveis se não forem devidamente protegidos, levando a ataques como sequestro de sessão e cross-site scripting.

Para implementar cookies seguros, os desenvolvedores devem seguir certos princípios bem documentados: por exemplo, usar o atributo “Secure” para todos os cookies, de modo a garantir que eles sejam enviados apenas por conexões HTTPS e protegidos contra interceptação por meio de espionagem; usar o atributo “HttpOnly” para impedir que os scripts do lado do cliente acessem cookies e proteger contra ataques de scripts entre sites; e usar o atributo “SameSite” para limitar quando os cookies são enviados a fim de mitigar ataques de falsificação de solicitações entre sites. Além disso, os cookies devem ter tempo de validade para restringir sua vida útil.

As técnicas de cookies seguros são fundamentais para mitigar vários ataques a aplicativos baseados na web, especialmente aqueles que visam o acesso não autorizado ou a manipulação dos cookies de sessão. Os desenvolvedores podem se defender contra ataques direcionados a eles ao empregar atributos específicos dentro dos cookies.

Análise do código estático

A análise do código estático é uma prática essencial de desenvolvimento de software. Envolve examinar o código-fonte para identificar possíveis vulnerabilidades, erros e práticas de codificação irregulares antes que o programa seja finalizado. Ao examinar o código em um estado “estático”, os desenvolvedores conseguem detectar e corrigir problemas no início do ciclo de vida do desenvolvimento. Essa é uma estratégia proativa que visa criar um software seguro, confiável e de alta qualidade.

As estratégias de segurança de aplicativos se concentram no desenvolvimento do software e nos ciclos de vida de implantação, com uma ênfase significativa em práticas de codificação seguras que incentivam os desenvolvedores a escrever códigos que evitem vulnerabilidades comuns, como injeção de SQL e scripts entre sites. As práticas de segurança de aplicações também exigem a realização de testes estáticos de segurança de aplicações (SAST) e testes dinâmicos de segurança de aplicações (DAST). As práticas de codificação desenvolvidas para acomodar patches e atualizações regulares são essenciais para a resolução imediata de vulnerabilidades recém-descobertas.

A análise do código estático dá respaldo à codificação segura e é realizada usando ferramentas especializadas, muitas vezes integradas em conjuntos de desenvolvimento de software. Essas ferramentas automatizam as verificações de código com regras pré-determinadas e sinalizam possíveis problemas que os desenvolvedores devem revisar e resolver.

Algumas ferramentas de análise estática amplamente utilizadas incluem SonarQube, Coverity e Fortify (<https://www.opentext.com/products/fortify-static-code-analyzer>), entre outras.

A análise do código estático no desenvolvimento de software é fundamental porque permite a detecção precoce de bugs e vulnerabilidades de segurança e ajuda a evitar falhas que poderiam ser catastróficas no produto final. Ela também melhora a qualidade e capacidade de manutenção do código ao impor padrões de codificação e práticas recomendadas.

Além disso, a análise do código estático ajuda a educar os desenvolvedores sobre erros comuns de codificação e riscos de segurança, o que ajuda a promover práticas de desenvolvimento conscientes da segurança.

Assinatura de código

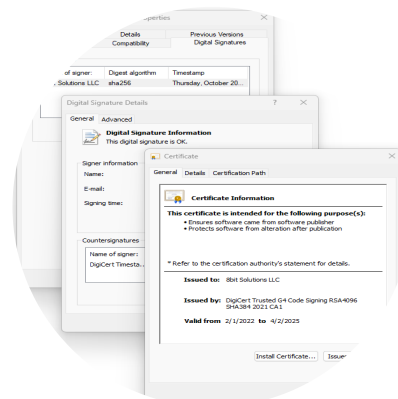
As práticas de [assinatura de código] utilizam assinaturas digitais para verificar a integridade e a autenticidade do código de software. A assinatura do código tem duas finalidades: garantir que o software não seja adulterado após a assinatura e confirmar a identidade do desenvolvedor do software.

Quando o software é assinado digitalmente, o signatário usa uma chave privada para criptografar um hash ou digitar o código. Esse hash criptografado e a identidade do signatário formam a assinatura digital. A assinatura do código requer o uso de um certificado emitido por uma autoridade de certificação (CA) confiável. O certificado contém informações sobre a identidade do signatário e é fundamental para verificar a assinatura digital. Se o certificado for válido e emitido por uma autoridade de certificação confiável, a identidade do desenvolvedor do software poderá ser verificada com segurança. A assinatura do código ajuda analistas e administradores a bloquear softwares não confiáveis e proteger os desenvolvedores de software, pois oferece um mecanismo de validação da autenticidade de seu código. De maneira geral, a assinatura do código contribui para criar confiança no processo de distribuição de software.

Embora a assinatura do código ofereça garantias sobre a origem dele e verifique a integridade do código, ela não garante inerentemente a segurança do código em si. A assinatura do código certifica a origem e a integridade do código, mas não avalia a qualidade ou a segurança deste. O código assinado ainda pode conter bugs, vulnerabilidades ou um código mal-intencionado inserido pelo autor original. A assinatura garante que o software provém do desenvolvedor esperado e no estado pretendido pelo desenvolvedor. Embora a assinatura do código seja um fator de confiança e autenticidade da distribuição de software, não se deve confiar nela para garantir um código seguro ou livre de bugs.

Figura 1. Análise da assinatura digital contida no programa de instalação do aplicativo Bitwarden Password

Management.



Descrição

A primeira janela está intitulada “Propriedades do Bitwarden-Installer – 2022.10.1”. A guia de assinaturas digitais está selecionada. Uma seção chamada "lista de assinaturas" fornece detalhes como nome do signatário, algoritmo de resumo e carimbo de data/hora.

A segunda janela está intitulada “detalhes da assinatura digital”. A guia geral está selecionada. A guia exibe informações do signatário e de contrassinaturas. A terceira janela está intitulada “Certificado”. A guia geral está selecionada. São exibidas informações do certificado. Os botões "Instalar certificado" e "Declaração do emissor" estão localizados no

canto inferior direito. Abaixo dos dois botões há um botão OK.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

[[Exposição de dados]] é uma falha que permite que informações privilegiadas (como um token, senha ou dado pessoal) sejam acessadas sem a devida aplicação dos controles de acesso apropriados. Os aplicativos devem transmitir esses dados apenas entre hosts autenticados, usando criptografia para proteger a sessão. Ao incorporar criptografia no código, é importante usar bibliotecas de criptografia padrão da indústria que sejam comprovadamente seguras, em vez de bibliotecas desenvolvidas internamente.

Tratamento de erros

Um aplicativo bem desenvolvido deve ser capaz de lidar com erros e [[exceções]] de forma apropriada. Isso significa que o aplicativo funcionará de forma controlada se algo imprevisível acontecer. Um erro ou exceção podem ser causados por uma entrada de usuário inválida, perda de conectividade de rede, falha de outro servidor ou do processo e assim por diante. Idealmente, o programador terá desenvolvido um [[manipulador estruturado de exceções (SEH)]] para definir as ações que o aplicativo deve executar a partir desse ponto. Cada procedimento pode ter vários gerenciadores de exceções.

Alguns gerenciadores tratam de erros e exceções esperados. Também deve haver um gerenciador geral para lidar com o inesperado. O objetivo principal deve ser que o aplicativo não falhe de forma a permitir que o invasor execute um código ou algum tipo de ataque de injeção. Um exemplo notório de manipulador de exceção mal implementado é o bug GoTo da Apple (<https://www.wired.com/2014/02/gotofail/> (<https://www.wired.com/2014/02/gotofail/>)).

Outro problema é que o interpretador de um aplicativos pode recorrer a um gerenciador padrão e exibir mensagens de erro padrão quando algo der errado. Isso pode revelar informações da plataforma e o funcionamento interno do código para um invasor. É melhor que um aplicativo use gerenciadores de erros personalizados para que o desenvolvedor possa escolher a quantidade de informações a exibir se ocorrer um erro.

Nota:

Tecnicamente, um erro é uma condição da qual o processo não consegue se recuperar, como o sistema ficar sem memória. Uma exceção é um tipo de erro que pode ser tratado por um bloco de código sem que o processo falhe. No entanto, observe que as exceções ainda assim podem gerar códigos ou mensagens de erro.

Gerenciamento da memória

Muitos ataques de código arbitrários dependem de procedimentos de gerenciamento de memória falhos no aplicativo. Isso permite que o invasor execute seu próprio código no espaço marcado pelo aplicativo. Existem práticas não seguras conhecidas para o gerenciamento da memória que devem ser evitadas. Há também verificações para processar entradas não confiáveis, como strings, de modo a garantir que não possam sobrescrever áreas de memória.

Client-Side vs. Validação no Lado do Servidor

Um aplicativo web (ou qualquer outro aplicativo cliente-servidor) pode ser criado para executar o código e validar a entrada localmente (no cliente) ou remotamente (no servidor). Um exemplo de execução do lado do cliente é um script de modelo de objeto de documento (DOM) para renderizar a página usando elementos dinâmicos da entrada do usuário. Os aplicativos podem usar ambas as técnicas para funções diferentes. O principal problema com a validação do lado do cliente é que este sempre estará mais vulnerável a algum tipo de malware que interfira no processo de validação. O principal problema com a validação do lado do servidor é que ela pode ser demorada, pois pode envolver várias transações entre o servidor e o cliente. Consequentemente, a validação do lado do cliente geralmente limita-se a informar ao usuário que há algum tipo de problema com a entrada antes de enviá-la ao servidor. Mesmo depois de passar na validação do lado do cliente, a entrada passa pela validação do lado do servidor antes de ser publicada (aceita). Confiar apenas na validação do lado do cliente é uma prática de programação insatisfatória.

Segurança de aplicativos na nuvem

O hardening da nuvem e a segurança de aplicativos são recursos complementares criados para respaldar o modelo de responsabilidade compartilhada em ambientes de nuvem, onde os provedores de serviços são responsáveis por proteger a infraestrutura, enquanto os clientes são responsáveis por proteger seus dados e aplicativos. As práticas de hardening da nuvem reforçam a infraestrutura dessa plataforma, reduzindo sua superfície de ataque, enquanto a segurança de aplicativos garante que o software seja criado, desenvolvido e implantado com segurança. Juntas, essas estratégias criam defesas em camadas para combater muitos tipos diferentes de ameaças.

O hardening da nuvem inclui políticas de acesso de privilégio mínimo para restringir os usuários às permissões mínimas necessárias para desempenhar suas funções. A criptografia protege os dados em movimento e em repouso. Auditorias regulares e práticas de monitoramento contínuo identificam possíveis riscos de segurança e avaliações regulares de vulnerabilidade e testes de penetração detectam e abordam possíveis problemas de segurança ou configurações incorretas.

Recursos de monitoramento

As práticas de codificação seguras visam principalmente a prevenção de vulnerabilidades dos softwares, mas também enfatizam as melhorias nos recursos de log e monitoramento. Esses recursos respaldam os analistas de segurança encarregados de detectar possíveis ameaças e atividades mal-intencionadas em softwares. Escrever código com

recursos aprimorados de monitoramento melhora o detalhamento e a eficácia dos sistemas de log e alerta, que são ferramentas fundamentais de monitoramento do sistema.

A implementação do registro em logs abrangente e significativo exige dos desenvolvedores um esforço para garantir aplicativos que gerem logs de eventos e atividades importantes, que respaldem auditorias de segurança, respostas a incidentes e soluções de problemas do sistema. As práticas de codificação seguras encorajam um tratamento eficaz de erros para ocultar ou mascarar informações confidenciais de depuração, e essa prática minimiza o risco de invasores explorarem as informações exibidas nas mensagens de erro. A integração de recursos de alerta em tempo real no código do aplicativo pode melhorar significativamente a detecção de ameaças. Por exemplo, o código que aciona alertas quando ocorrem eventos específicos, como repetidas tentativas de login malsucedidas ou transferências de dados incomuns, ajuda os analistas de segurança a monitorar os aplicativos com mais eficiência. Esses alertas geralmente indicam uma possível violação de segurança e concedem informações importantes para as equipes de resposta a incidentes.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

[[Sandboxing]] é um mecanismo de segurança utilizado no desenvolvimento e na operação de software para isolar processos em execução uns dos outros ou impedir que eles acessem o sistema no qual estão sendo executados. Uma sandbox é um recurso de proteção criado para controlar um programa de modo que ele funcione com acesso altamente restritivo. Essa estratégia de contenção reduz o impacto em potencial de softwares mal-intencionados ou com mau funcionamento, por isso é eficaz tanto para melhorar a segurança e estabilidade do sistema como para atenuar os riscos associados ao software.

Um exemplo prático de sandbox foi implementado em navegadores modernos, como o Google Chrome, que separa cada aba e extensão em processos distintos. Se um site ou extensão do navegador em uma aba do navegador tentar executar um código mal-intencionado, ele estará confinado à sandbox dessa aba. Essa ação evita que códigos mal-intencionados afetem todo o navegador ou o sistema operacional subjacente. Da mesma forma, se uma aba falhar, isso não fará com que todo o navegador falhe, o que melhora a confiabilidade.

Os sistemas operacionais também utilizam a sandbox para isolar os aplicativos. Por exemplo, o iOS e o Android usam a sandbox para limitar as ações de cada aplicativo. O aplicativo em uma sandbox consegue acessar seus próprios dados e recursos, mas não consegue acessar outros dados do aplicativo ou outros recursos não essenciais do sistema sem permissão explícita. Essa estratégia limita os danos causados por aplicativos malfeitos ou mal-intencionados.

Máquinas virtuais (VMs) e contêineres como o Docker são outro exemplo de sandbox em maior escala. Cada VM ou contêiner pode ser executado isoladamente, separado do host e de seus semelhantes. Outras VMs ou outros contêineres permanecem inalterados se um sofrer uma violação de segurança ou falha no sistema.

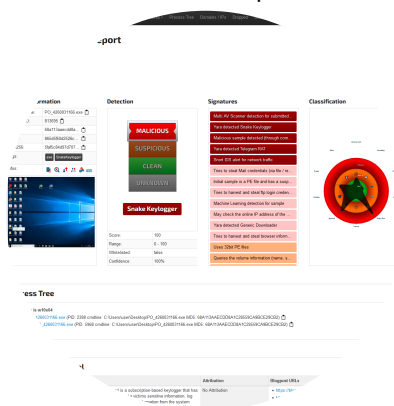
Áreas restritas em operações de segurança

As ferramentas de sandbox são fundamentais nas operações e análises de segurança, principalmente na detecção e compreensão de atividades de malware por meio de inspeção forense. As ferramentas de sandbox criam um ambiente fechado e controlado que permite a execução segura (também chamada de detonação) de softwares possivelmente prejudiciais sem comprometer a integridade do ambiente de TI.

Alguns exemplos dessas ferramentas incluem o Cuckoo Sandbox, um sistema de código aberto que executa arquivos em um ambiente isolado e examina seu comportamento, registrando atividades essenciais, como chamadas de sistema e tráfego de rede.

Outra ferramenta importante é o Joe Sandbox, que não requer configuração ou instalação no ambiente da organização, mas pode ser acessado por meio de um navegador web. O Joe Sandbox utiliza várias técnicas de análise, incluindo aprendizado de máquina, para examinar o software.

Figura 1. Análise do Joe Sandbox de um arquivo mal-intencionado executável.



Captura de tela de cortesia de Joe Security, LLC.

Descrição

A guia de visão geral no topo está selecionada. A tela exibe o relatório de análise do Windows. Uma seção intitulada "visão geral" fornece dados sobre informações gerais, detecção, assinaturas e classificação. Outras duas seções são intituladas "árvore de processos" e "inteligência sobre ameaças de malware".



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Você deve ser capaz de configurar protocolos seguros para acesso e gerenciamento de rede local, serviços de aplicativos e acesso e gerenciamento remotos.

Diretrizes para aprimorar os recursos de segurança de aplicativos

Siga estas diretrizes ao implementar protocolos de rede:

- Avalie os requisitos para proteger os protocolos de aplicativos, como a necessidade de certificados ou chaves para autenticação e uso da porta TCP/UDP:
 - Implante certificados em servidores web para usar com HTTPS.
 - Implante certificados em servidores de email para usar com SMTP, POP3 e IMAP seguros.
 - Implante certificados ou chaves de host em servidores de arquivos para usar com FTPS ou SFTP.
 - Implante certificados em clientes de email para usar com o S/MIME.

Siga estas diretrizes para melhorar projetos de desenvolvimento de aplicativos:

- Treine os desenvolvedores em técnicas de codificação seguras para fornecer proteção específica contra ataques:
 - Entenda diferentes categorias de ataques a aplicativos web que exploram códigos vulneráveis.
 - Identifique ataques de injeção (XSS, SQL, XML, LDAP) que exploram a falta de validação de entrada.
 - Repita e solicite ataques de falsificação que exploram a falta de mecanismos seguros de gerenciamento de sessões.
- Anlise e teste o código usando análise estática e dinâmica, prestando atenção especial a validação de entrada, codificação de saída, tratamento de erros e exposição de dados.



This content is only available on larger screen sizes. Please revisit this page on a larger device.